

Relatório de Laboratório

Laboratório de Experimentação de Software - Lab02 - Assistentes de IA vs. codificação manual

Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Turno / Período	Noite / 6º
Professor(a)	Danilo Maia
Laboratório	Lab02 - Assistentes de IA vs. codificação manual
Grupo (trio)	Enzo Casaes · Cauê Moraes · Leonardo Viana
Link do repositório / GitHub Projects	Repositório: https://github.com/Enzocasaes/Laboratorio-medicao-experimentacao-de-software GitHub Projects: https://github.com/users/Enzocasaes/projects/2
Data de entrega	23/09/2026

Laboratório de Experimentação de Software - Lab02

1. Introdução

Ferramentas de IA generativa para programação, como GitHub Copilot, ChatGPT, Claude e Gemini, tornaram-se comuns no desenvolvimento de software. Apesar dessa adoção, grande parte do debate sobre ganhos de produtividade e qualidade ainda se apoia em relatos individuais. Este laboratório busca produzir evidência controlada e reproduzível por meio de um experimento que compara a resolução de katas de programação com e sem assistência de IA.

O estudo avalia três dimensões: produtividade, medida pelo tempo até uma solução passar em todos os testes de aceitação; qualidade funcional, observada pela taxa de testes de aceitação aprovados; e qualidade estrutural, avaliada por complexidade ciclomática, duplicação de código e métricas de apoio. O experimento adota desenho crossover/within-subject, com time-box máximo de 35 minutos por trial.

Questões de Pesquisa

RQ1 - O uso de assistente de IA reduz o tempo necessário para resolver uma tarefa de programação?

RQ2 - O uso de assistente de IA reduz a quantidade de defeitos, operacionalizada pelos testes que falham no código produzido?

RQ3 - O uso de assistente de IA altera a complexidade ciclomática ou a duplicação do código

produzido?

Hipóteses definidas antes da análise dos dados

RQ1: H0 - não existe diferença no tempo até verde entre com-ia e sem-ia; H1 - existe diferença entre os tratamentos.

RQ2: H0 - não existe diferença na taxa de sucesso funcional entre os tratamentos; H1 - existe diferença.

RQ3: H0 - não existe diferença nas métricas estruturais entre os tratamentos; H1 - existe diferença. A RQ3 foi desdobrada em duas sub-hipóteses independentes: **H3a** (complexidade ciclomática média por função) e **H3b** (percentual de linhas duplicadas).

A família de hipóteses submetida à correção de Holm foi fixada no desenho, antes da coleta, em quatro elementos: H1, H2, H3a e H3b. Métricas de apoio à RQ3 (LOC, Índice de Manutenibilidade, número de funções e complexidade máxima) foram mantidas deliberadamente fora dessa família, para evitar ampliar o conjunto de testes depois de observar os dados.

Como contribuição adicional do grupo, foram incluídos um instrumento próprio de métricas para JavaScript com testes automatizados, um protocolo de baixa indexação dos katas para reduzir o risco de memorização, e uma análise estatística complementar com tamanho de efeito e correção de Holm. O Índice de Manutenibilidade e o número de prompts foram mantidos como métricas exploratórias.

Laboratório de Experimentação de Software - Lab02

2. Contexto

Este é o laboratório dedicado ao processo de resolução de katas com e sem IA. O objeto de estudo não é um repositório externo, mas o próprio processo experimental: estudantes de graduação resolvem tarefas de dificuldade comparável em condições controladas, alternando o tratamento com-ia e sem-ia. A análise é orientada pelo GQM (Goal-Question-Metric): o objetivo é comparar o efeito do assistente de IA sobre tempo de resolução, defeitos funcionais e estrutura do código.

Foram definidos seis katas autorais em JavaScript, com 12 testes de aceitação automatizados por kata e solução de referência entre 42 e 51 LOC. O uso de exercícios autorais e pouco indexados reduz a ameaça de o assistente apenas reproduzir uma solução conhecida. O desenho prevê 3 participantes x 6 katas = 18 trials, sendo 9 com-ia e 9 sem-ia. A coleta foi concluída integralmente: os 18 trials foram executados pelos três participantes, e todos os seis katas formam pares completo para o teste de Wilcoxon.

Laboratório de Experimentação de Software - Lab02

3. Metodologia

O experimento utiliza desenho crossover/within-subject, contrabalanceado por matriz integrante x kata. Cada participante resolve seis katas individualmente, metade com assistente de IA e metade

sem assistente, em ordem contrabalanceada definida por quadrado latino. O time-box é de 35 minutos por trial. Um trial que não atingir todos os testes dentro desse limite permanece na base como censurado em 2100 segundos, evitando descarte seletivo.

3.1 Principais Desafios

O primeiro desafio foi construir katas de dificuldade comparável e, ao mesmo tempo, pouco suscetíveis à memorização pelo modelo. Para isso, o grupo utilizou exercícios autorais e realizou uma checagem de baixa indexação, com busca textual e teste de reconhecimento direto pelo assistente.

Outro desafio foi preservar a comparabilidade entre os tratamentos. O mesmo assistente deve ser utilizado em todos os trials com-ia, enquanto nos trials sem-ia o assistente e o autocomplete de IA permanecem desativados. A coleta também depende de cronometragem uniforme e de uma regra única para trials incompletos.

Por fim, a linguagem escolhida foi JavaScript, para a qual CK e Radon não se aplicam diretamente. Isso exigiu a construção de um instrumento próprio para coletar complexidade ciclomática, duplicação, LOC e Índice de Manutenibilidade de modo consistente e reprodutível.

3.2 Tomadas de Decisão

Assistente: Claude via claude.ai, plano gratuito, mantido como tratamento único nos trials com ia. Versão exata do modelo na data da coleta: Claude Opus 5.5.

Linguagem: JavaScript com Node.js ≥ 18 e testes de aceitação via node:test. **Katas:** seis exercícios autorais, com 12 testes de aceitação cada, para reduzir diferenças de cobertura e risco de memorização.

Laboratório de Experimentação de Software - Lab02

Time-box: 35 minutos por trial; trials sem sucesso devem ser registrados como censurados em 2100 s e nunca descartados. Nenhum trial atingiu o limite.

Estatística descritiva: mediana e IQR, por serem mais robustos a outliers em amostras pequenas.

Estatística inferencial: Wilcoxon signed-rank pareado por kata, bilateral, alfa = 0,05, com p exato por enumeração, correção de Holm sobre as quatro hipóteses da família e tamanho de efeito r reportado sempre.

Outliers: mantidos e discutidos, sem remoção automática. A detecção por $IQR \times 1,5$ não acusou outliers nas métricas estáticas;

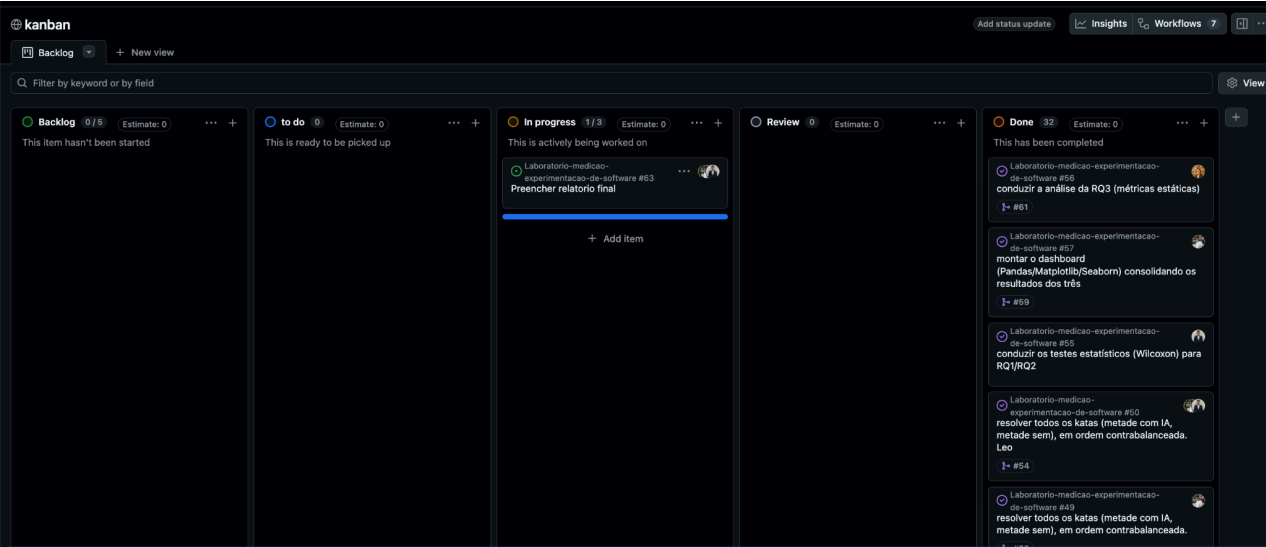
Processo no GitHub Projects: colunas mínimas Backlog -> To Do -> Doing -> Review -> Done. Limite de WIP da coluna Doing: 3 pois somos 3 integrantes e cada um poderia estar executando apenas uma task por vez.

3.3 Etapas

Sprint/Etapa	Entregas	Responsável(is)	Issues
S01 - Desenho	Hipóteses; variáveis; tratamentos; desenho crossover; ameaças à validade; configuração do board.	Enzo Casaes Leonardo Viana Cauê Moraes	#45
S02 - Preparação	Definição/validação dos 6 katas; testes de aceitação; cronômetro; scripts de métricas; ambiente e protocolo de baixa indexação.	Enzo Casaes Leonardo Viana Cauê Moraes	#45, #46, #49
S03 - Execução	Execução dos trials com-ia e sem-ia; registro de tempo, testes, observações e código final. 18/18 concluídos.	Enzo Casaes (P1) Cauê Moraes (P2) Leonardo Viana (P3)	#49, #50, #51
S03 - Análise	mediana/IQR; Wilcoxon; correção de Holm; análise da RQ3; dashboard de resultados.	Cauê Moraes Leonardo Viana Enzo Casaes	#55, #56, #57
Relatório Final	Discussão por RQ, ameaças à validade, conclusão, reprodutibilidade e entrega do dashboard.	Grupo	#63

Configuração do processo

O board deve refletir o fluxo real do grupo com Backlog, To Do, Doing, Review e Done, além da política de WIP adotada.



3.4 Ferramentas

Ferramenta	Uso no experimento
VS Code	IDE utilizada nos trials. Versão de cada integrante: 1.128.1
Claude / claude.ai	Assistente utilizado nos trials com-ia. Versão exata do modelo: Claude Opus 5.5.
Node.js >= 18	Execução dos katas, testes e scripts de coleta/análise.
node:test	Testes automatizados de aceitação: 12 por kata.
cronometro.js	Cronometragem única do time-to-green e registro do trial.
preparar-trial.js	Monta a pasta do trial (enunciado, suíte e esqueleto) antes de iniciar o cronômetro.
metricas.js	Instrumento próprio: McCabe médio, duplicação, LOC e MI.
analise.js	Medianas/IQR, Wilcoxon exato, tamanho de efeito e correção de Holm.
dashboard.py	Dashboard e visualizações em Pandas + Matplotlib/Seaborn.
Git / GitHub / GitHub Projects v2	Versionamento, rastreabilidade dos artefatos e gestão do processo.

3.5 Tabela de Métricas

RQ	Métrica	Definição Operacional	Unidade	Ferramenta / Fonte
----	---------	-----------------------	---------	--------------------

RQ1	Time-to-Green	Tempo entre início do trial e instante em que 100% dos testes passam; se não houver sucesso até 35 min, registrar 2100 s e marcar censurado.	segundos	cronometro.js
RQ1	Nº de prompts	Contagem de interações enviadas ao assistente durante o trial com-ia.	contagem	registro do trial
RQ2	Taxa de sucesso	testes_passando / testes_totais ao final do trial.	%	node:test
RQ2	Testes falhando	testes_totais - testes_passando ao final do time-box.	contagem	node:test
RQ3a	Complexidade ciclomática média	Média da complexidade de McCabe por função do código final: 1 + if/for/while/do/case/catch/ /??/ternário. Funções aninhadas contam à parte.	índice	metricas.js
RQ3b	Duplicação	Percentual de linhas significativas cobertas por janelas de 5 linhas repetidas, sem atravessar fronteira de arquivo.	%	metricas.js
RQ3	LOC	Linhas não vazias que não são comentário. Métrica de controle obrigatória ao reportar complexidade ou duplicação.	linhas	metricas.js
RQ3	Maintainability Index	Índice composto normalizado 0-100 a partir de complexidade, LOC e volume de Halstead; uso exploratório.	índice	metricas.js
RQ3	Nº de funções	Funções detectadas no código final (declaração, expressão, arrow e método). Leitura de apoio.	contagem	metricas.js

3.6 Inovações Propostas pelo Grupo (30% da nota)

1) Instrumento próprio de métricas para JavaScript.

Como CK é voltado a Java e Radon a Python, o grupo implementou metricas.js em Node puro para manter a linguagem dos katas em JavaScript. O instrumento parte de um tokenizador que descarta comentários, strings, template strings e literais de regex antes de qualquer contagem, de modo que um if dentro de uma string não seja contado como desvio de fluxo. Mede McCabe,

Laboratório de Experimentação de Software - Lab02
duplicação, LOC e MI, e é coberto por testes automatizados, aumentando a reprodutibilidade do

laboratório.

2) Protocolo de baixa indexação e reconhecimento.

Além de produzir katas autorais, o grupo verificou se os enunciados eram facilmente localizáveis e se o assistente reconhecia diretamente os exercícios. Nenhum dos seis foi reconhecido no protocolo aplicado, o que reduz - sem eliminar - a ameaça de memorização.

3) Análise estatística complementar.

Além do Wilcoxon solicitado, o grupo calcula o p exato por enumeração dos 2^n sinais possíveis, em vez da aproximação normal, que é ruim com n pequeno; reporta tamanho de efeito r; e aplica correção de Holm sobre as quatro hipóteses da família fixada no desenho. O script também informa o menor p atingível para o n disponível, tornando explícito quando o teste não teria como rejeitar H_0 qualquer que fosse o resultado.

4) Métricas exploratórias e de apoio.

O Índice de Manutenibilidade e o número de prompts aprofundam a discussão. Para a RQ3, o grupo acrescentou LOC como controle obrigatório e o número de funções como leitura de apoio, mantidos fora da correção de Holm justamente para não ampliar a família de hipóteses depois de ver os dados. Como mostra a seção 4.3, é esse conjunto que permite interpretar corretamente o resultado estrutural.

4. Resultados

4.1 Coleta de Dados

O desenho prevê 18 trials: 3 participantes x 6 katas, com 9 com-ia e 9 sem-ia. A coleta foi concluída integralmente: Enzo (P1) 6/6, Cauê (P2) 6/6 e Leonardo (P3) 6/6. A estatística descritiva usa $n=9$ por tratamento, e a análise inferencial pareada usa os seis katas, todos com observações nos dois tratamentos.

Nenhum trial foi censurado e nenhum ultrapassou o time-box de 35 minutos. A maior duração observada foi 23:28. Todos os 18 trials encerraram com 12/12 testes de aceitação passando. A duplicação de código foi 0% em todos os trials, criando efeito de teto em RQ2 e efeito de piso em duplicação.

A detecção de outliers por IQR x 1,5 não acusou nenhum valor extremo nas métricas estáticas. Em tempo, o trial caue_kata-03_com-ia (355 s) foi mantido: o registro informa que os testes já haviam passado por volta de 266 s, mas a marcação de encerramento ocorreu depois, por dúvida de procedimento. O dado foi mantido e discutido em vez de removido.

Trials do participante P1 (Enzo)

Ordem	Kata	Tratamento	Tempo até verde	Testes
1	kata-01	com-ia	00:18	12/12
2	kata-04	sem-ia	10:21	12/12
3	kata-02	com-ia	00:16	12/12
4	kata-05	sem-ia	08:17	12/12
5	kata-03	com-ia	00:18	12/12
6	kata-06	sem-ia	14:26	12/12

Trials do participante P2 (Cauê)

Ordem	Kata	Tratamento	Tempo até verde	Testes
1	kata-06	sem-ia	19:06	12/12
2	kata-03	com-ia	05:55	12/12
3	kata-01	sem-ia	14:42	12/12
4	kata-04	com-ia	01:10	12/12
5	kata-02	sem-ia	23:28	12/12
6	kata-05	com-ia	01:29	12/12

Trials do participante P3 (Leonardo)

Ordem	Kata	Tratamento	Tempo até verde	Testes
1	kata-02	sem-ia	09:31	12/12
2	kata-05	com-ia	02:18	12/12
3	kata-03	sem-ia	23:05	12/12
4	kata-06	com-ia	02:51	12/12
5	kata-04	sem-ia	12:24	12/12
6	kata-01	com-ia	03:02	12/12

4.2 Visualização Gráfica

RQ1 - O uso de assistente de IA reduz o tempo necessário para resolver uma tarefa de programação?

Laboratório de Experimentação de Software - Lab02

Tempo até verde por kata, pareado com-ia × sem-ia (unidade de análise do Passo 4)

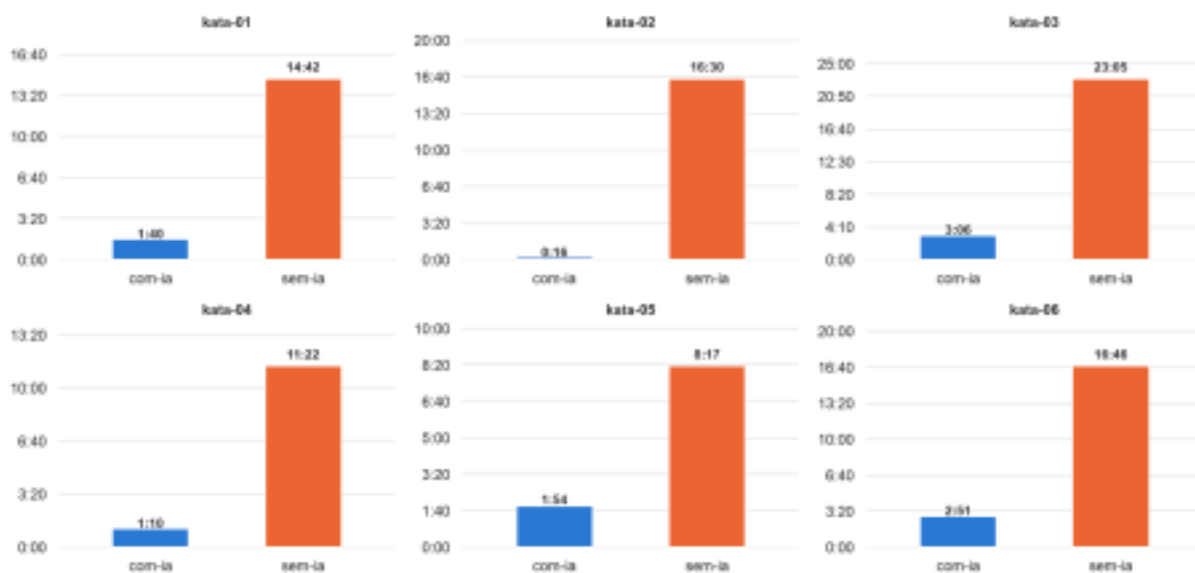


Figura 1 - Comparação pareada do tempo até verde nos seis katas, agora com os dois tratamentos disponíveis.

Nos seis pares completos, o tratamento com-ia foi mais rápido em todos os casos. Na análise agregada, a mediana foi 01:29 com-ia (IQR 00:18-02:57) e 14:26 sem-ia (IQR 09:56-21:06).

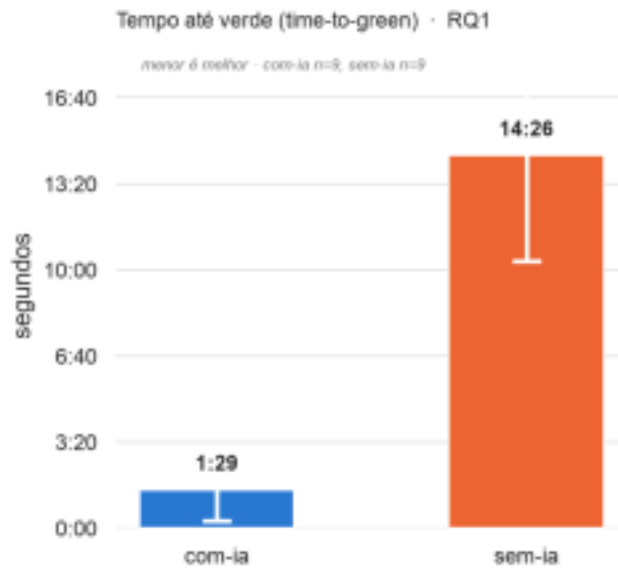


Figura 2 - Mediana e IQR do time-to-green por tratamento.

RQ2 - O uso de assistente de IA reduz a quantidade de defeitos no código produzido?

Laboratório de Experimentação de Software - Lab02



Figura 3 - Taxa de sucesso dos testes de aceitação por tratamento.

A taxa de sucesso foi de 100% em ambos os tratamentos, nos 18 trials. Esta métrica não diferenciou com-ia e sem-ia.

RQ3 - O uso de assistente de IA altera a complexidade ciclomática ou a duplicação do código produzido?

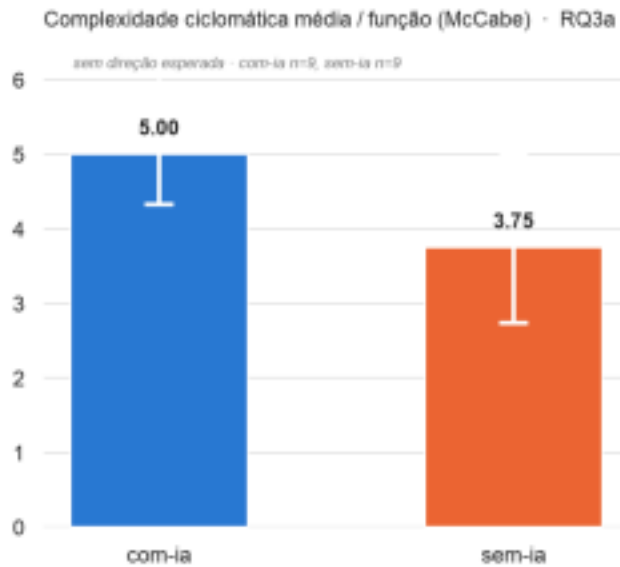


Figura 4 - Complexidade ciclomática média por função.

Laboratório de Experimentação de Software - Lab02



Figura 5 - Duplicação de código.

A complexidade ciclomática mediana foi 5,00 com-ia (IQR 4,29-9,25) e 3,75 sem-ia (IQR 2,63-6,25). A duplicação foi 0% nos dois tratamentos. Com a coleta completa, a direção passou a ser

consistente: cinco dos seis pares apontam maior complexidade com IA (o único contrário é o kata 02), e o sinal se repete nos três participantes independentemente (P1 +1,50; P2 +5,00; P3 +1,25).

Métricas de controle e aprofundamento

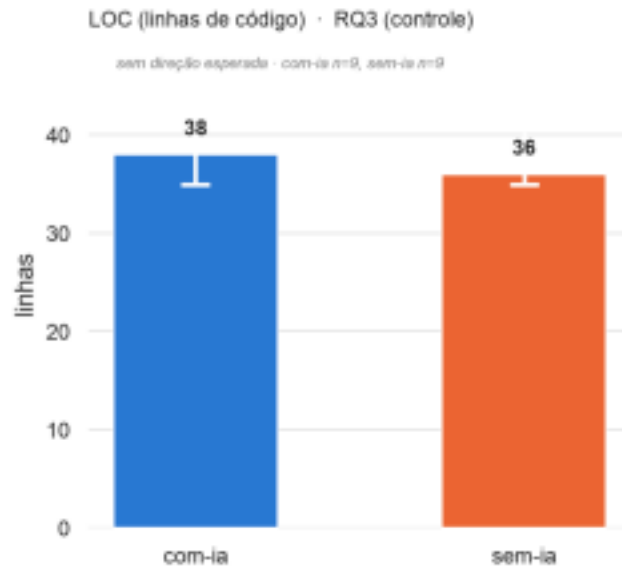


Figura 6 - LOC como métrica de controle.

Laboratório de Experimentação de Software - Lab02

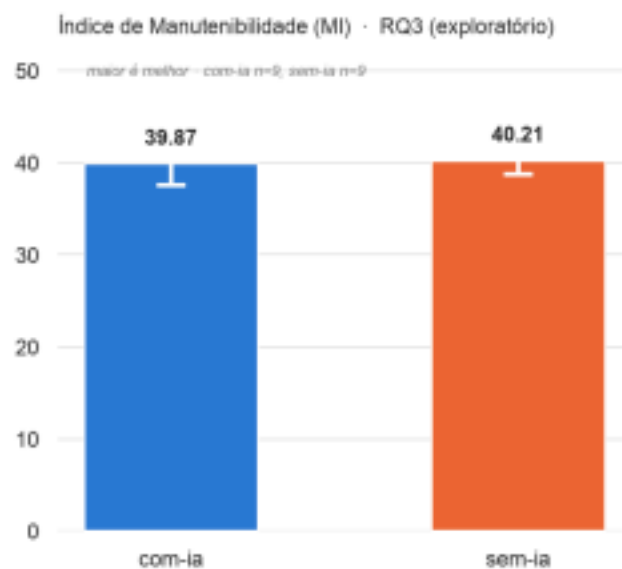


Figura 7 - Índice de Manutenibilidade.

O LOC mediano foi 38 com-ia e 36 sem-ia, com $r = 0,043$ e $p = 1,000$: **o tamanho do código é equivalente entre os tratamentos**. Esse controle é o que dá sentido ao resultado de complexidade

— a diferença observada não vem de verbosidade. O MI mediano foi 39,87 com-ia e 40,21 sem-ia, praticamente empatado, porque o índice composto compensa a maior complexidade com o LOC equivalente.

A métrica de apoio que fecha a interpretação é o número de funções: mediana 3 com-ia contra 4 sem-ia, com $r = 0,601$ e diferença negativa nos seis katas e nos três participantes. Mesmo LOC, menos funções e maior complexidade por função descrevem código menos decomposto, não código mais longo.

Laboratório de Experimentação de Software - Lab02

4.3 Discussão

A Tabela 3 resume a análise inferencial com a coleta completa. O Wilcoxon usa $n=6$ pares, um por kata, já que todos os seis têm observações nos dois tratamentos. Em RQ2 e em duplicação todas as diferenças são zero, portanto o teste não é aplicável por ausência de variância.

RQ	Métrica	n pares	Mediana com-ia	Mediana sem-ia	Wilcoxon / efeito	Decisão
RQ1	Tempo até verde	6	89 s	866 s	$V=0$; $p=0,031$; $p_{Holm}=0,063$; $r=0,899$	Não rejeita H_0
RQ2	Taxa de sucesso	6 (dif.=0)	1,00	1,00	Indisponível: sem variância	Teste não aplicável
RQ3a	Complexidade	6	5,00	3,75	$V=4$; $p=0,219$; $p_{Holm}=0,219$; $r=0,556$	Não rejeita H_0
RQ3b	Duplicação	6 (dif.=0)	0%	0%	Indisponível: sem variância	Teste não aplicável

RQ3	LOC (controle)	6	38	36	$r=0,043$; $p=1,000$ (fora de Holm)	Sem diferença
RQ3	MI (exploratório)	6	39,87	40,21	$r=0,043$; $p=1,000$ (fora de Holm)	Sem diferença
RQ3	Nº de funções (apoio)	6	3	4	$r=0,601$; $p=0,188$ (fora de Holm)	Sem diferença

RQ1.

A hipótese de menor tempo com IA é fortemente apoiada descritivamente: os seis pares apresentaram tempo menor com-ia, sem uma única exceção, com mediana das diferenças de -808,5 s (cerca de 13,5 minutos) e tamanho de efeito muito grande ($r = 0,899$). O p exato bilateral é 0,031, ou seja, o mínimo atingível com seis pares e abaixo de 0,05. Entretanto, após a correção de Holm sobre a família de quatro hipóteses, o p ajustado sobe para 0,063 e não cruza o limiar. Seguindo a regra fixada no desenho, que manda concluir pelo p ajustado, não se rejeita H_0 . A leitura correta é que o efeito é grande e perfeitamente consistente, mas o desenho com seis pares não tem poder para sustentá-lo depois do controle de múltiplas comparações.

Vale registrar o contraste com o corte anterior de 12 trials, em que o mesmo teste dispunha de quatro pares e o menor p atingível era 0,125 — ali nem em princípio seria possível chegar à significância. Completar a coleta levou o p bruto exatamente ao limite do que o desenho permite.

Laboratório de Experimentação de Software - Lab02

RQ2.

A expectativa de redução de defeitos não foi confirmada nem refutada estatisticamente, porque os dois tratamentos atingiram 100% de sucesso nos 18 trials. O resultado é um efeito de teto: não houve espaço para a métrica discriminar os grupos. O dado permite afirmar somente que a vantagem temporal observada em RQ1 não veio acompanhada de queda na taxa de sucesso funcional — o que, em si, já é informativo: a aceleração não custou qualidade funcional.

RQ3.

A complexidade apresentou mediana maior com-ia (5,00 contra 3,75), com direção consistente em cinco dos seis pares e tamanho de efeito grande ($r = 0,556$), mas o Wilcoxon não fornece evidência para rejeitar H_0 (p Holm = 0,219). A duplicação foi 0% nos dois tratamentos e não pôde ser testada. O achado relevante aparece quando as três métricas são lidas juntas, como o enunciado exige: com LOC equivalente ($r = 0,043$) e menos funções no código assistido (mediana 3 contra 4, $r = 0,601$, diferença negativa em todos os pares), o quadro descrito é o de código menos decomposto, e não de código mais verboso. A complexidade máxima por trial segue o mesmo sentido (10 contra 9, $r = 0,472$).

Nenhum desses valores atinge significância com seis pares, e por isso nada aqui é conclusivo. Mas os quatro sinais — complexidade média, complexidade máxima, número de funções e LOC — são coerentes entre si e se repetem entre participantes, o que caracteriza uma hipótese bem formada para réplica, e não ruído.

Sobre a duplicação, o efeito de piso tem explicação no próprio objeto experimental: soluções de 24 a 79 LOC, de função única, praticamente não comportam um bloco repetido de cinco linhas. A métrica não discrimina neste contexto. Uma réplica precisaria de katas maiores, janela menor que cinco linhas, ou detecção por token em vez de linha, como faz o PMD CPD.

Inovações.

O instrumento próprio tornou possível manter JavaScript e coletar as métricas exigidas de forma automatizada e testada. O protocolo de baixa indexação reduz uma ameaça específica do uso de IA. A correção de Holm e o tamanho de efeito são o que permite interpretar RQ1 com honestidade: há uma diferença observada enorme e perfeitamente consistente, que ainda assim não sobrevive ao controle de múltiplas comparações com este n . E as métricas de apoio da RQ3, mantidas fora da família de Holm, são o que transforma um resultado negativo isolado em uma interpretação estrutural defensável.

Ameaças à validade.

Na validade interna, permanecem o efeito de aprendizado e a familiaridade desigual com a IA, além de quatro desvios registrados: o uso de Claude Code em vez de claude.ai nos três trials com-ia de P3, que quebra a ferramenta única do tratamento; as consultas de procedimento ao assistente durante três trials sem-ia; a abertura accidental de uma solução de referência antes de caue_kata 04_com-ia; e o encerramento tardio do cronômetro em leo_kata-03_sem-ia, corrigido por registro manual. Registra-se ainda que P3 executou os seis trials em uma janela de

Laboratório de Experimentação de Software - Lab02

aproximadamente 1h23, contrariando o limite de três trials por sessão previsto no desenho para conter fadiga.

Na validade de construto, testes de aceitação são apenas uma proxy de defeitos, e as métricas estruturais não representam toda a qualidade do código; o MI, por ser composto, chega a mascarar o padrão que as métricas isoladas revelam. Na validade externa, a amostra é reduzida, composta por estudantes e katas pequenos, o que limita a generalização para equipes profissionais e bases legadas. Na validade de conclusão, o principal risco segue sendo o baixo poder: com seis pares, o menor p bilateral atingível é 0,031, de modo que só um efeito perfeitamente consistente poderia ser detectado, e ainda assim a correção de Holm o absorve. Quanto à memorização, os katas autorais e o protocolo de reconhecimento reduzem, mas não eliminam, essa possibilidade.

Laboratório de Experimentação de Software - Lab02

5. Conclusão

Com a coleta completa dos 18 trials, a condição com-ia apresentou tempo até verde substancialmente menor: mediana de 89 s contra 866 s, uma diferença mediana de aproximadamente 13,5 minutos. A direção se repetiu nos seis katas pareados, sem exceção, com tamanho de efeito muito grande ($r = 0,899$) e p exato de 0,031. Ainda assim, após a correção de Holm o p ajustado é 0,063 e a hipótese nula não é rejeitada pelo critério fixado no desenho. RQ1 apresenta, portanto, um sinal de produtividade forte e consistente, sem significância estatística sob controle de múltiplas comparações.

Para RQ2, os dois tratamentos obtiveram 100% de sucesso nos testes de aceitação em todos os 18 trials, de modo que não foi observada diferença funcional. O resultado é um efeito de teto, mas permite afirmar que o ganho de tempo não veio acompanhado de perda de qualidade funcional.

Para RQ3, a complexidade ciclomática apresentou mediana maior com-ia (5,00 contra 3,75), agora com direção consistente em cinco dos seis katas e efeito grande ($r = 0,556$), porém sem diferença estatisticamente sustentada ($p_{\text{Holm}} = 0,219$); a duplicação permaneceu em 0% nos dois tratamentos e não pôde ser testada. Lida junto do controle de LOC, que ficou equivalente, e do número de funções, menor com IA em todos os pares, a evidência descritiva sugere que o assistente produziu código menos decomposto, e não mais verboso — um padrão coerente, porém ainda não demonstrado estatisticamente.

As principais limitações são o pequeno número de pares, a utilização de estudantes e katas curtos, os desvios de protocolo registrados na seção 4.3 e as ameaças de aprendizado e memorização. Para uma réplica, o grupo recomenda ampliar o número de participantes — o que aumenta o n de pares e o poder do Wilcoxon —, usar katas maiores e com mais de um módulo, de modo que a duplicação deixe de ter efeito de piso, tarefas com dificuldade suficiente para evitar o teto de 100% em RQ2, e tarefas de manutenção em bases de código reais, mais próximas do uso profissional do assistente.

6. Referências

BASIL, Victor R.; CALDIERA, Gianluigi; ROMBACH, H. Dieter. The Goal Question Metric Approach. In: Encyclopedia of Software Engineering. Wiley, 1994.

ZUSE, Horst. A framework of software measurement. Walter de Gruyter, 2013.

WILCOXON, Frank. Individual comparisons by ranking methods. Biometrics Bulletin, 1945.

HOLM, Sture. A simple sequentially rejective multiple test procedure. Scandinavian Journal of Statistics, 1979.

McCABE, Thomas J. A Complexity Measure. IEEE Transactions on Software Engineering, 1976.

Laboratório de Experimentação de Software - Lab02

WOHLIN, Claes et al. Experimentation in Software Engineering. Springer, 2012. Vídeo

Repositório do experimento: <https://github.com/Enzocasaes/Laboratorio-medicao-experimentacao>

de-software